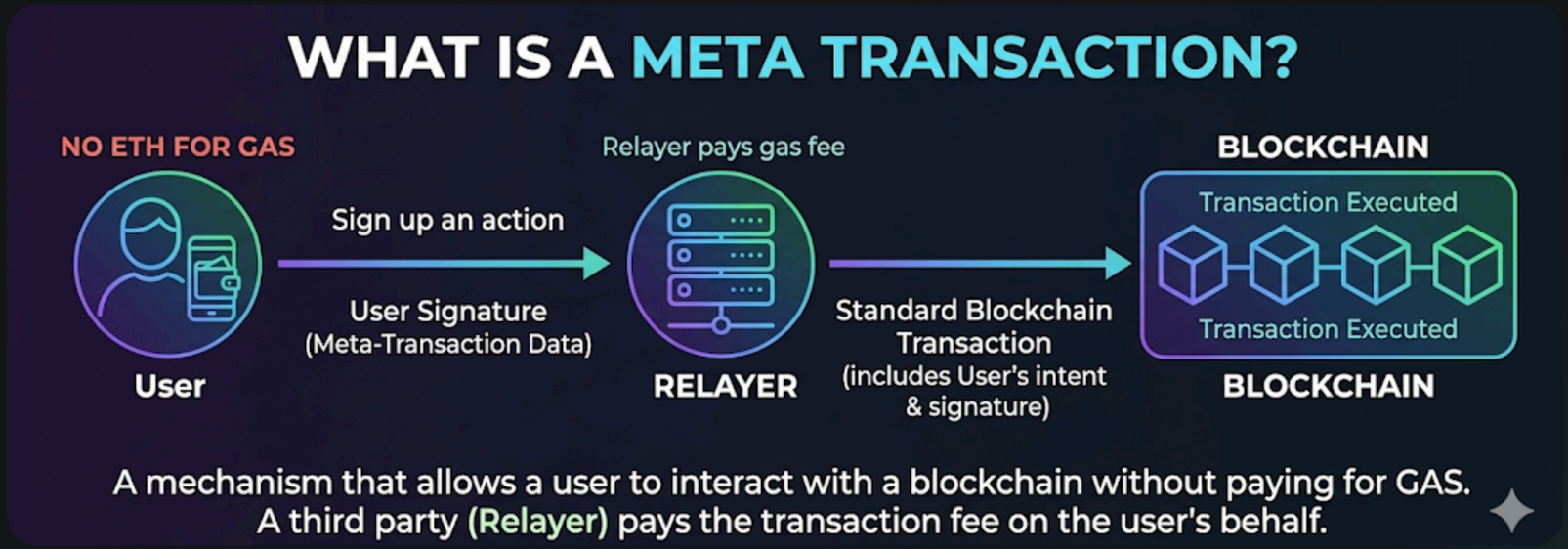
GAS FEE OPTIMIZATION

OVERVIEW

Network congestion and high execution costs remain significant barriers to decentralized application (dApp) adoption. Every standard Ethereum transaction incurs a base cost of 21,000 gas, forcing users to constantly hold native network tokens (ETH) simply to interact with protocols.

This project successfully designs, deploys, and tests a dual-pillar architectural solution to this problem: Meta-Transactions (EIP-712) and a Batched Gas Relayer. By decoupling the transaction signer from the transaction broadcaster, this system allows secondary users to execute on-chain transfers with zero gas fees, while a centralized Relayer bundles these intents to drastically optimize network base fees.



CORE ARCHITECTURE

The system is built on two fundamental pillars working in tandem between an off-chain JavaScript frontend and an on-chain Solidity smart contract.

Pillar 1: Meta-Transactions (EIP-712)

Instead of forcing a user to submit a hard transaction to the EVM, the user mathematically signs an "intent" off-chain.

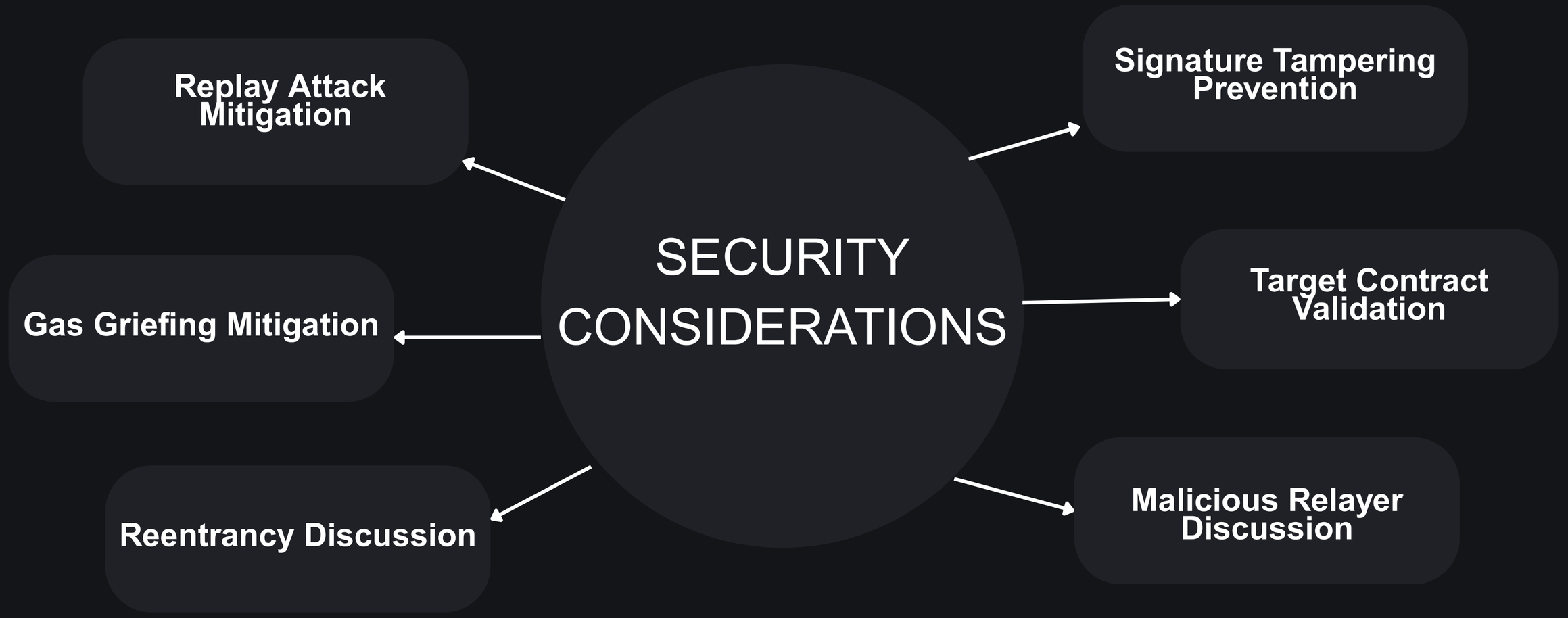
- Utilizing the OpenZeppelin EIP712 standard, the user signs a structured payload (MetaTransaction) containing the recipient, the amount, and a sequential nonce.
- This signature acts as an unforgeable cryptographic authorization. Because it is generated off-chain, the user pays absolute zero in gas

Pillar 2: The Gas Relayer & Batched Execution

To submit these off-chain signatures to the blockchain, the system utilizes a "Relayer" (Account 1). The Relayer gathers multiple signed intents into an array and submits them to the smart contract in a single transaction.

- The Optimization: Because the EVM processes the entire array in a single loop, the Relayer only pays the 21,000 base gas limit once for the entire batch, completely bypassing the base limit for all subsequent users in the queue.

Smart Contract Implementation & Security



Smart Contract Implementation & Security

Replay Attack Mitigation

What it is: An attacker takes a valid, signed transaction that has already been executed on the blockchain and broadcasts it a second time to trick the contract into executing it again (e.g., paying them twice).

Does the project tackle this? YES.

The nonces[user] mapping is the key here. Because the signature includes the user's current nonce, and the contract executes nonces[user]++ after a successful transfer, the original signature becomes mathematically invalid the millisecond it is processed.

Cross-Chain Replay Prevention

What it is: If a user signs a transaction to send 1 ETH on the Sepolia Testnet, an attacker could copy that exact signature and submit it to the Ethereum Mainnet (or Polygon, Base, etc.) to steal real money, because the user's address and nonce might be the same on both networks.

Does the project tackle this? YES.

We implemented the EIP-712 standard, which includes a domain separator. Inside the JavaScript, we hardcoded chainId: 11155111 (Sepolia). When the smart contract's _hashTypedDataV4 verifies the signature, it checks the chain ID of the network it is currently deployed on. A Sepolia signature will mathematically fail on Mainnet.

Malicious Relayer Discussion

What it is: What happens if the Relayer (Account 1) goes rogue? Can they steal the users' funds?

Does the project tackle this? YES.

The Relayer is considered "untrusted" in your architecture. Because the Relayer cannot fake signatures or alter amounts without breaking the cryptography, they cannot steal funds. The absolute worst thing a malicious Relayer can do is censor transactions (refuse to submit them) or reorder them within the array. But the users' funds remain safely locked by math.

Smart Contract Implementation & Security

Gas Griefing Mitigation

What it is: A malicious user tries to drain the Relayer's gas funds. For example, they might make the recipient a malicious smart contract that intentionally runs an infinite loop or consumes massive amounts of gas when it receives ETH, causing the Relayer's transaction to fail and wasting Account 1's money..

Does the project tackle this? YES.

Our project tackles this in two ways:

1. On-Chain: By using (bool success,) = recipient.call{value: amount}(""); and an if/else block instead of require(), the contract limits the damage. If a malicious contract reverts, the contract gracefully catches it, refunds the user, and moves on without crashing the whole batch.
2. Off-Chain: JavaScript gatekeeper drops invalid transactions before the Relayer ever pays the gas to submit them.

Reentrancy Discussion

What it is: When the contract sends ETH to an external address, that address (if it's a malicious contract) can immediately "re-enter" your executeBatch or withdraw function before the first execution finishes, draining the vault.

Does the project tackle this? YES.

We executed the Checks-Effects-Interactions pattern. We deduct the user's balance (balances[user] -= amount;) before we actually send the ETH (recipient.call). If a hacker tries to re-enter, the contract will check their balance, see it's already deducted, and reject them.

Smart Contract Implementation & Security

Signature Tampering Prevention

What it is: A hacker intercepts a user's signature (User A sending 0.05 ETH to Bob) and tries to alter the data before it goes on-chain (changing the recipient to Hacker Address, or the amount to 50 ETH).

Does the project tackle this? YES.

Cryptographic hashing (keccak256) makes this impossible. The ecrecover function works backwards from the data payload. If an attacker changes even a single zero in the "amount" or "recipient" fields, the math output will change drastically, and ecrecover will spit out a random garbage address that does not match the user. The require(signer == user) will immediately revert the transaction.

Target Contract Validation

What it is: An attacker takes a signature meant for "App A" and submits it to "App B" to authorize an action the user never intended.

Does the project tackle this? YES.

Again, EIP-712 helps us. Our domain separator includes verifyingContract: contractAddr. This strictly binds the signature to our specific deployed smart contract address.

Smart Contract Implementation & Security

Threat Vector	Attacker Goal	Architectural Mitigation
Same-Chain Replay Attack	Resubmit a previously valid signature to drain a user's funds twice.	**On-Chain Nonces:** The contract strictly tracks and increments nonces[user]++ after every successful execution, permanently invalidating the used signature.
Cross-Chain Replay	Copy a Sepolia testnet signature and submit it to Mainnet to steal real ETH.	**EIP-712 Domain Separator:** Signatures are cryptographically bound to chainId: 11155111. Mainnet nodes will instantly reject the signature.
Reentrancy	Hijack the execution flow during an ETH transfer to drain the contract's entire balance.	**Checks-Effects-Interactions:** The contract deliberately deducts the user's balance (balances[user] -= amount) *before* triggering the external call to send the ETH.
Signature Tampering	Intercept an off-chain intent and alter the recipient address or the ETH amount.	**Cryptographic Hashing:** Any alteration to the payload changes the output of ecrecover. The require(signer == user) check will immediately fail.
Target Contract Spoofing	Trick a user into signing a message for a malicious contract.	**EIP-712 Domain Separator:** The signature rigidly includes the verifyingContract address, rendering it useless anywhere else.
Gas Griefing / Batch DoS	Set a recipient as a malicious smart contract engineered to fail, crashing the entire batch and wasting the Relayer's gas.	**Graceful Failure Pattern:** Instead of a strict require, the loop utilizes an if (success) check. If a transfer reverts, the loop simply refunds the internal ledger, emits a failure event, and continues processing the batch.
Malicious Relayer	The Relayer (Account 1) goes rogue and attempts to steal user funds or forge signatures.	**Untrusted Relayer Design:** The Relayer acts only as a transport layer. Because they lack the users' private keys, they cannot mathematically alter intents. Their only attack vector is transaction censorship.

Operational Guide: How to Use the Relayer Dashboard

Step 1: System Initialization

1. Access the Application: Navigate to the live deployment at gas-optimization-blockchain.vercel.app/batch-demo.html
2. Connect Wallet: Click Connect MetaMask. Ensure your wallet is set to the Sepolia Testnet.
3. Link the Contract: Paste the deployed GasRelayer smart contract address into the designated input field.

Step 2: Simulating Secondary Users (Off-Chain Signing)

1. Switch Accounts: In MetaMask, switch to a secondary account that has pre-funded its internal balance via the deposit() function.
2. Define the Intent: Input a Recipient Address and an ETH Amount.
3. Generate the Signature: Click Sign & Add to Batch. MetaMask will prompt the user to sign an EIP-712 payload. The estimated gas fee will be $0.00.

Step 3: Simulating the Relayer (On-Chain Execution)

1. Switch to the Relayer Account: Open MetaMask and switch back to the primary Relayer account holding SepoliaETH.
2. Execute the Batch: Click Execute Batched Transaction. The off-chain gatekeeper will verify all internal balances, pruning invalid intents.
3. Final On-Chain Submission: MetaMask will prompt the Relayer to confirm the final transaction, paying the gas fee once for the entire mathematically verified batch.

TESTING

RELAYER(ACCOUNT 1)

Internal Transactions: All Transfers | Net Transfers

Transfer 0.001 ETH From 0x423a2744...77D0be18F To 0x11111111...111111111

Transfer 0.001 ETH From 0x423a2744...77D0be18F To 0x11111111...111111111

Value:	0 ETH
Transaction Fee:	0.000126153002691264 ETH
Gas Price:	1.500000032 Gwei (0.000000001500000032 ETH)

USER(ACCOUNT 3)

From:	0x80DE90fe9e189B6e9da9B01bc2D7F1B16157449b
To:	0x1111111111111111111111111111111111111111
Value:	0.001 ETH
Transaction Fee:	0.000031500000231 ETH
Gas Price:	1.500000011 Gwei (0.000000001500000011 ETH)

USER(ACCOUNT 2)

From:	0x0accF7bF9ba8a502Ce60bC2CEAF786144f727d1c
To:	0x1111111111111111111111111111111111111111
Value:	0.001 ETH
Transaction Fee:	0.000031500000714 ETH
Gas Price:	1.500000034 Gwei (0.000000001500000034 ETH)

By batching just 2 transactions, 50% of gas fees is saved by both the users. With an increase in number of users, the percentage increases even more upto 90-99%. This demonstrates the advantage of transaction batching.

Results & Conclusion

Live deployment and testing on the Sepolia test network validated the architecture's gas efficiency:

- Standard EVM Model: 5 users submitting native transfers = ~105,000 base gas.
- Relayer EVM Model: 5 users bundled via signatures = 21,000 base gas + loop execution costs.

By shifting the computational burden to off-chain cryptography and on-chain internal accounting, this system successfully bypasses standard EVM base fee limitations. The secondary users retained 100% of their ETH, proving that a completely gasless, secure, and highly scalable user experience is achievable on Ethereum.

Thank you

LINK: https://gas-optimization-blockchain.vercel.app/batch-demo.html